EXP NO: 10	MINI PROJECT - HEART DISEASE PREDICTION SYSTEM

AIM:

To develop an end-to-end machine learning project that predicts the presence of heart disease from 13 patient attributes, covering data exploration, preprocessing, model comparison, hyperparameter tuning, evaluation and deployment as a REST API.

ALGORITHM:

1. Import the required libraries and load the heart disease dataset (13 patient attributes and a target label). A simulated dataset with the same attributes is generated when heart.csv is not available.
2. Explore the data: shape, missing values, class distribution and correlation heatmap.
3. Split the data into training (80%) and testing (20%) sets using stratified sampling.
4. Build a Pipeline for every model with median imputation (missing values), StandardScaler (feature scaling) and the classifier, so the same preprocessing is applied at prediction time.
5. Compare Logistic Regression, Decision Tree, Random Forest, Gradient Boosting and SVM using 5-fold cross-validation (ROC-AUC).
6. Select the best model and tune its hyperparameters using GridSearchCV.
7. Evaluate the final model on the unseen test set using accuracy, precision, recall, F1 score, ROC-AUC, confusion matrix, ROC curve and feature importance.
8. Save the trained pipeline using joblib and deploy it as a Flask REST API with GET / and POST /predict routes.
9. Test the API with a high-risk patient, a low-risk patient and an incomplete input.

CODE:

```
# ===== MINI PROJECT: HEART DISEASE PREDICTION SYSTEM =====
# 1. Import libraries
import json, threading, time, logging
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib, requests
```

```
from flask import Flask, request, jsonify
from werkzeug.serving import make_server
from sklearn.model_selection import train_test_split, cross_val_score,
    GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
    f1_score,
                             roc_auc_score, roc_curve, confusion_matrix,
                                 classification_report)

# 2. Load the dataset
# Uses heart.csv if it is uploaded (UCI Heart Disease attributes). Otherwise a
# simulated dataset with the same 13 attributes is generated.
try:
    df = pd.read_csv("heart.csv")
    print("Loaded heart.csv")
except FileNotFoundError:
    rng = np.random.default_rng(42)
    n = 1000
    age = rng.normal(54, 9, n).clip(29, 77).round()
    sex = rng.binomial(1, 0.68, n)
    cp = rng.choice([0, 1, 2, 3], n, p=[0.47, 0.17, 0.28, 0.08])
    trestbps = rng.normal(131, 17, n).clip(94, 200).round()
    chol = rng.normal(246, 51, n).clip(126, 564).round()
    fbs = rng.binomial(1, 0.15, n)
    restecg = rng.choice([0, 1, 2], n, p=[0.50, 0.48, 0.02])
    thalach = (205 - 0.9 * age + rng.normal(0, 15, n)).clip(71, 202).round()
    exang = rng.binomial(1, 0.33, n)
    oldpeak = rng.gamma(1.2, 0.9, n).clip(0, 6.2).round(1)
    slope = rng.choice([0, 1, 2], n, p=[0.07, 0.46, 0.47])
    ca = rng.choice([0, 1, 2, 3], n, p=[0.58, 0.22, 0.13, 0.07])
    thal = rng.choice([1, 2, 3], n, p=[0.06, 0.55, 0.39])
    z = 1.8 * (-2.6 + 0.04 * (age - 54) + 0.8 * sex + 0.7 * (cp == 0) + 0.012 *
        (trestbps - 131)
         + 0.004 * (chol - 246) - 0.03 * (thalach - 150) + 1.0 * exang + 0.6 *
             oldpeak
         + 0.5 * ca + 0.7 * (thal == 3))
    target = rng.binomial(1, 1 / (1 + np.exp(-z)))
    flip = rng.random(n) < 0.04                         # 4% label noise
    target = np.where(flip, 1 - target, target)
    df = pd.DataFrame(dict(age=age, sex=sex, cp=cp, trestbps=trestbps,
        chol=chol, fbs=fbs,
                           restecg=restecg, thalach=thalach, exang=exang,
                               oldpeak=oldpeak,
                           slope=slope, ca=ca, thal=thal, target=target))
    df.loc[rng.random(n) < 0.03, "chol"] = np.nan     # a few missing values
    df.loc[rng.random(n) < 0.02, "ca"] = np.nan
    print("heart.csv not found - using simulated dataset with UCI Heart Disease
        attributes")
```

```
# 3. Explore the data
pd.set_option("display.max_columns", None); pd.set_option("display.width", 120)
print("\nShape:", df.shape)
print("\nFirst 5 records (transposed for readability):\n", df.head().T)
print("\nMissing values:\n", df.isnull().sum()[df.isnull().sum() > 0])
print("\nClass distribution (0 = No disease, 1 = Disease):\n",
    df["target"].value_counts())

plt.figure(figsize=(13, 5))
plt.subplot(1, 2, 1)
sns.countplot(x="target", data=df, palette="Set2"); plt.title("Class
    Distribution")
plt.subplot(1, 2, 2)
sns.heatmap(df.corr(), cmap="coolwarm", center=0, cbar_kws={"shrink": 0.8})
plt.title("Correlation Heatmap")
plt.tight_layout(); plt.show()

# 4. Split data (imputation + scaling are inside the pipeline, so there is no
    data leakage)
X = df.drop("target", axis=1)
y = df["target"]
feature_names = list(X.columns)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("\nTrain shape:", X_train.shape, " Test shape:", X_test.shape)

# 5. Compare several models
def make_pipe(clf):
    return Pipeline([("imputer", SimpleImputer(strategy="median")),
                     ("scaler", StandardScaler()), ("clf", clf)])

models = {
    "Logistic Regression": make_pipe(LogisticRegression(max_iter=1000)),
    "Decision Tree": make_pipe(DecisionTreeClassifier(max_depth=5,
        random_state=42)),
    "Random Forest": make_pipe(RandomForestClassifier(n_estimators=100,
        random_state=42)),
    "Gradient Boosting": make_pipe(GradientBoostingClassifier(random_state=42)),
    "SVM": make_pipe(SVC(probability=True, random_state=42)),
}
rows = []
for name, pipe in models.items():
    cv = cross_val_score(pipe, X_train, y_train, cv=5, scoring="roc_auc")
    rows.append([name, cv.mean(), cv.std()])
cmp_df = pd.DataFrame(rows, columns=["Model", "CV ROC-AUC (mean)", "Std"]) \
           .sort_values("CV ROC-AUC (mean)",
               ascending=False).reset_index(drop=True)
print("\nModel comparison (5-fold cross-validation):\n")
print(cmp_df.round(4).to_string())
best_name = cmp_df.loc[0, "Model"]
print("\nBest model selected for tuning:", best_name)

plt.figure(figsize=(8, 4))
plt.barh(cmp_df["Model"], cmp_df["CV ROC-AUC (mean)"], xerr=cmp_df["Std"],
    color="steelblue")
```

```
plt.xlim(0.5, 1.0); plt.gca().invert_yaxis(); plt.xlabel("CV ROC-AUC")
plt.title("Model Comparison"); plt.tight_layout(); plt.show()

# 6. Hyper-parameter tuning of the best model
grids = {
    "Logistic Regression": {"clf__C": [0.01, 0.1, 1, 10]},
    "Decision Tree": {"clf__max_depth": [3, 5, 7]},
    "Random Forest": {"clf__n_estimators": [100, 200], "clf__max_depth": [None,
        5, 10],
                      "clf__min_samples_split": [2, 5]},
    "Gradient Boosting": {"clf__n_estimators": [100, 200],
        "clf__learning_rate": [0.05, 0.1],
                          "clf__max_depth": [2, 3]},
    "SVM": {"clf__C": [0.1, 1, 10], "clf__gamma": ["scale", 0.01]},
}
grid = GridSearchCV(models[best_name], grids[best_name], cv=5,
    scoring="roc_auc", n_jobs=-1)
grid.fit(X_train, y_train)
final_model = grid.best_estimator_
print("Best parameters:", grid.best_params_)
print("Best CV ROC-AUC:", round(grid.best_score_, 4))

# 7. Final evaluation on the unseen test set
y_pred = final_model.predict(X_test)
y_prob = final_model.predict_proba(X_test)[:, 1]
print("\n=== FINAL TEST RESULTS ===")
print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(y_test, y_pred), 4))
print("Recall   :", round(recall_score(y_test, y_pred), 4))
print("F1 Score :", round(f1_score(y_test, y_pred), 4))
print("ROC-AUC  :", round(roc_auc_score(y_test, y_prob), 4))
print("\nClassification Report:\n", classification_report(
    y_test, y_pred, target_names=["No Disease", "Disease"]))

clf = final_model.named_steps["clf"]
if hasattr(clf, "feature_importances_"):
    imp = pd.Series(clf.feature_importances_, index=feature_names)
else:
    imp = pd.Series(np.abs(clf.coef_[0]), index=feature_names)
imp = imp.sort_values(ascending=False)
print("Top 5 important features:\n", imp.head(5).round(4))

fig, ax = plt.subplots(1, 2, figsize=(11, 4.5))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d",
    cmap="Blues", ax=ax[0],
            xticklabels=["No Disease", "Disease"], yticklabels=["No Disease",
                "Disease"])
ax[0].set_title("Confusion Matrix"); ax[0].set_xlabel("Predicted");
    ax[0].set_ylabel("Actual")
fpr, tpr, _ = roc_curve(y_test, y_prob)
ax[1].plot(fpr, tpr, label=f"AUC = {roc_auc_score(y_test, y_prob):.3f}")
ax[1].plot([0, 1], [0, 1], "k--"); ax[1].set_title("ROC Curve")
ax[1].set_xlabel("False Positive Rate"); ax[1].set_ylabel("True Positive
    Rate"); ax[1].legend()
plt.tight_layout(); plt.show()
```

```
plt.figure(figsize=(7, 4))
imp.head(8).sort_values().plot(kind="barh", color="teal")
plt.title("Top Feature Importance"); plt.xlabel("Importance")
plt.tight_layout(); plt.show()

# 8. Save the trained model
joblib.dump(final_model, "heart_model.pkl")
print("\nModel saved as heart_model.pkl")

# 9. Deploy as a REST API using Flask
logging.getLogger("werkzeug").setLevel(logging.ERROR)
app = Flask(__name__)
model = joblib.load("heart_model.pkl")

@app.route("/", methods=["GET"])
def home():
    return jsonify({"message": "Heart Disease Prediction API is running",
                    "usage": "POST /predict with JSON of the 13 patient
                        features"})

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    missing = [f for f in feature_names if not data or f not in data]
    if missing:
        return jsonify({"error": "Missing features", "missing": missing}), 400
    patient = pd.DataFrame([data])[feature_names]
    prob = float(model.predict_proba(patient)[0][1])
    return jsonify({"prediction": int(prob >= 0.5),
                    "result": "Heart Disease Likely" if prob >= 0.5 else "No
                        Heart Disease",
                    "probability": round(prob, 4)})

server = make_server("127.0.0.1", 5000, app)
thread = threading.Thread(target=server.serve_forever); thread.start();
    time.sleep(1)
print("API started at http://127.0.0.1:5000")

# 10. Test the API with two patients
high_risk = dict(age=63, sex=1, cp=0, trestbps=160, chol=286, fbs=1, restecg=1,
    thalach=108,
                 exang=1, oldpeak=2.6, slope=0, ca=3, thal=3)
low_risk = dict(age=38, sex=0, cp=2, trestbps=115, chol=190, fbs=0, restecg=0,
    thalach=178,
                exang=0, oldpeak=0.0, slope=2, ca=0, thal=2)
print("\nGET / ->", requests.get("http://127.0.0.1:5000/").json())
for name, p in [("High-risk patient", high_risk), ("Low-risk patient",
    low_risk)]:
    r = requests.post("http://127.0.0.1:5000/predict", json=p)
    print(f"{name} ->", r.status_code, r.json())
r = requests.post("http://127.0.0.1:5000/predict", json={"age": 50})
print("Incomplete input ->", r.status_code, r.json()["error"])

server.shutdown(); thread.join()
print("API stopped.")
```

OUTPUT:

```
heart.csv not found - using simulated dataset with UCI Heart Disease attributes

Shape: (1000, 14)

First 5 records (transposed for readability):
                0      1      2      3      4
age          57.0   45.0   61.0   62.0   36.0
sex           1.0    1.0    1.0    1.0    1.0
cp            2.0    3.0    0.0    1.0    2.0
trestbps    119.0  166.0  130.0  148.0  130.0
chol        193.0  214.0  193.0  166.0  257.0
fbs           1.0    0.0    0.0    0.0    0.0
restecg       0.0    1.0    0.0    0.0    1.0
thalach     145.0  175.0  182.0  125.0  160.0
exang         0.0    0.0    1.0    0.0    0.0
oldpeak       1.4    1.8    0.7    4.2    0.1
slope         1.0    2.0    1.0    1.0    2.0
ca            0.0    0.0    0.0    3.0    1.0
thal          2.0    3.0    2.0    3.0    2.0
target        0.0    1.0    0.0    0.0    0.0

Missing values:
 chol    27
ca      29
dtype: int64

Class distribution (0 = No disease, 1 = Disease):
 target
0    589
1    411
Name: count, dtype: int64

Train shape: (800, 13)  Test shape: (200, 13)

Model comparison (5-fold cross-validation):

                  Model  CV ROC-AUC (mean)     Std
0  Logistic Regression             0.8365  0.0321
1                  SVM             0.8197  0.0281
2        Random Forest             0.7820  0.0415
3    Gradient Boosting             0.7671  0.0391
4        Decision Tree             0.6322  0.0313

Best model selected for tuning: Logistic Regression
Best parameters: {'clf__C': 10}
Best CV ROC-AUC: 0.8366

=== FINAL TEST RESULTS ===
Accuracy : 0.8
Precision: 0.7917
Recall   : 0.6951
F1 Score : 0.7403
ROC-AUC  : 0.8532
```

```
Classification Report:
               precision    recall  f1-score   support

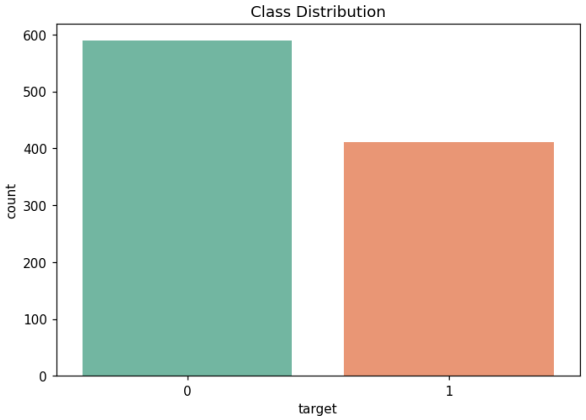
  No Disease       0.80      0.87      0.84       118
     Disease       0.79      0.70      0.74        82

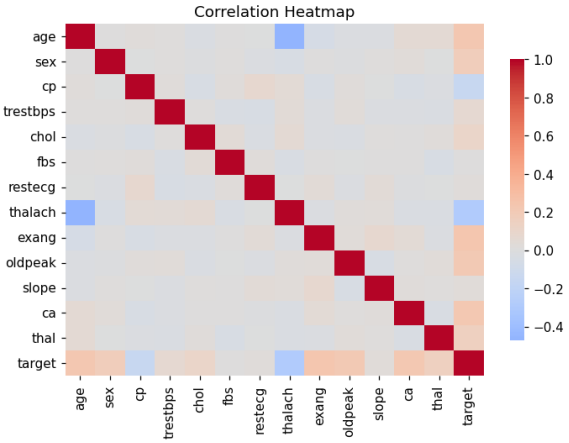
    accuracy                           0.80       200
   macro avg       0.80      0.78      0.79       200
weighted avg       0.80      0.80      0.80       200

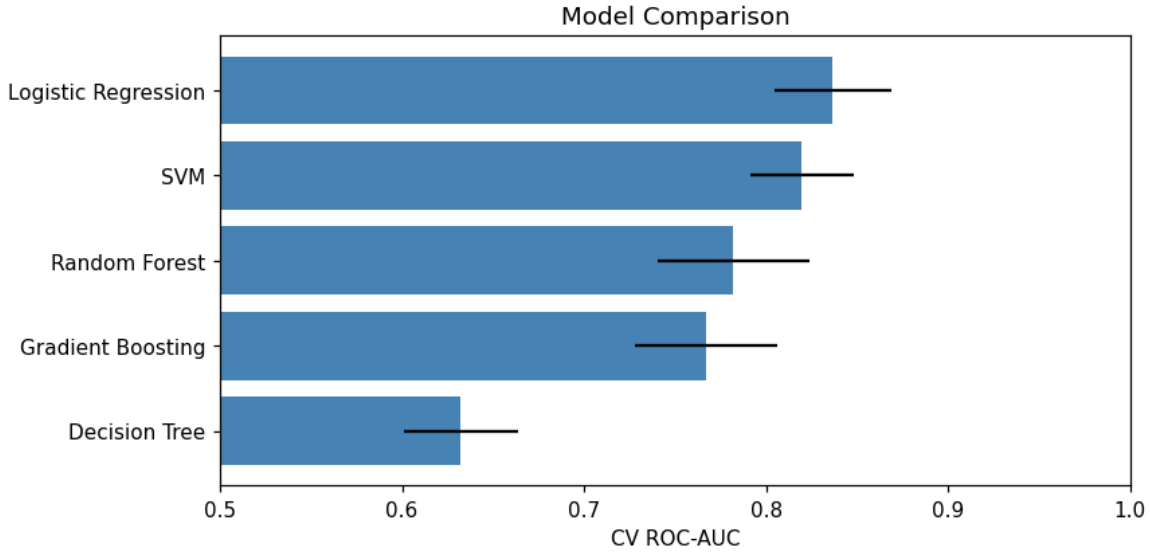
Top 5 important features:
 thalach    0.6596
oldpeak    0.6468
exang      0.6238
sex        0.4760
thal       0.4643
dtype: float64

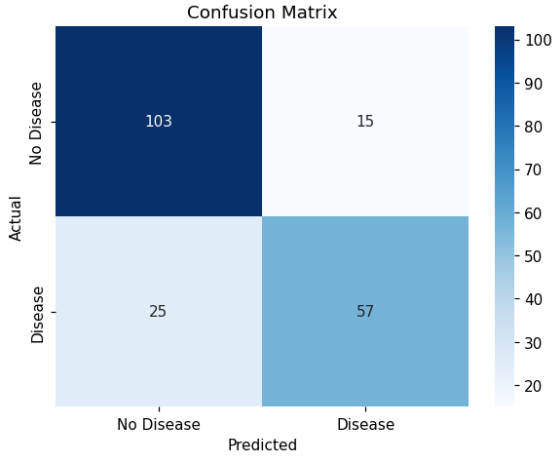
Model saved as heart_model.pkl
API started at http://127.0.0.1:5000

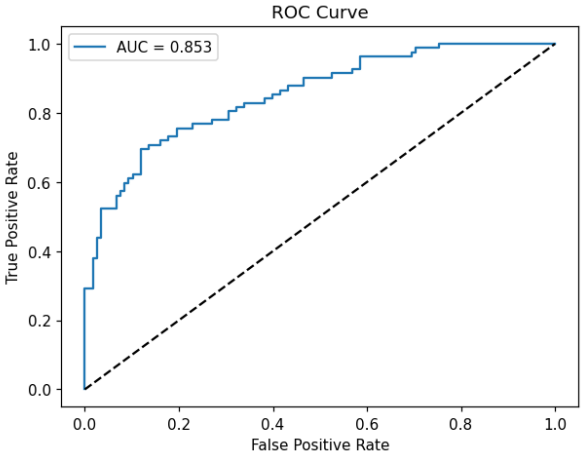
GET / -> {'message': 'Heart Disease Prediction API is running', 'usage': 'POST
    /predict with JSON of the 13 patient features'}
High-risk patient -> 200 {'prediction': 1, 'probability': 0.9988, 'result':
    'Heart Disease Likely'}
Low-risk patient -> 200 {'prediction': 0, 'probability': 0.0036, 'result': 'No
    Heart Disease'}
Incomplete input -> 400 Missing features
API stopped.
```

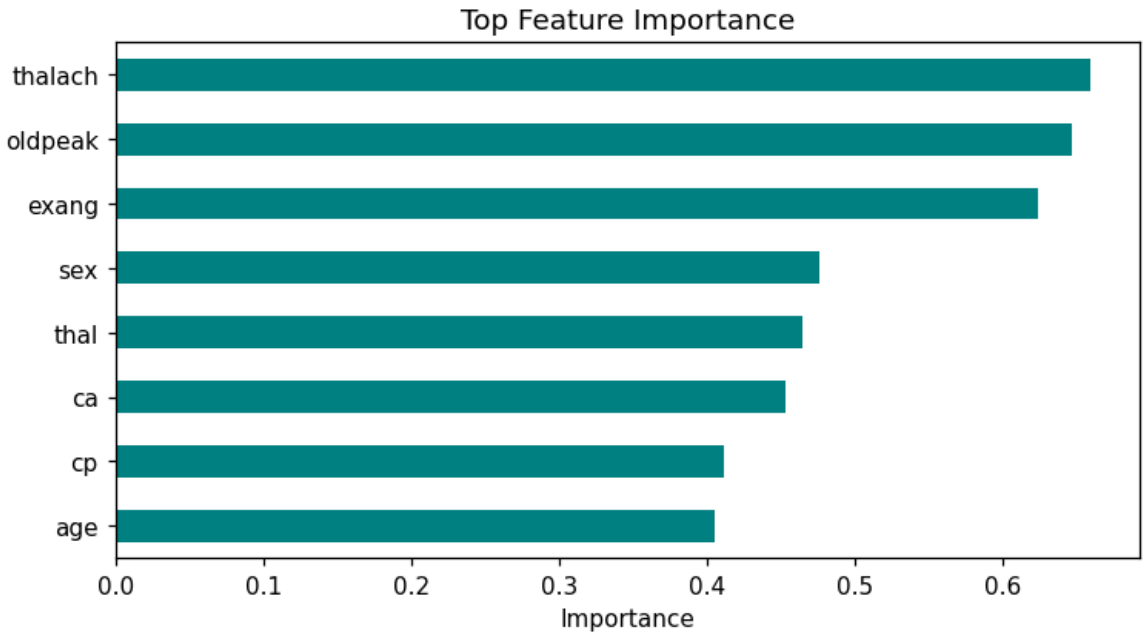





Model Comparison
Logistic Regression
SVM
Random Forest
Gradient Boosting
Decision Tree
0.5
0.6
0.7
0.8
0.9
1.0
CV ROC-AUC


Confusion Matrix
No Disease
Disease
Actual
103
15
25
57
No Disease
Disease
Predicted
100
90
80
70
60
50
40
30
20


ROC Curve
AUC = 0.853
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0



RESULT:

The Heart Disease Prediction System was successfully developed and deployed. Logistic Regression gave the best cross-validation ROC-AUC (0.8365) and, after tuning (C=10), achieved a test accuracy of 80.0%, F1 score of 0.74 and ROC-AUC of 0.853. Thalach (maximum heart rate), oldpeak and exang were the most influential features. The trained model was served through a Flask REST API that correctly classified a high-risk and a low-risk patient and rejected incomplete input with status code 400. Note: this run used a simulated dataset with the UCI attributes, so the numbers will differ on the real dataset.